

Relatório Trabalho Final – Asteroids 3D

Felipe Gabriel Soares Rodrigues - 20249020884¹, Luanderson Oliveira de Carvalho
- 20249012050¹

¹Departamento de Computação – Universidade Federal do Piauí (UFPI)

{felipe.soares, luanderson.carvalho}@ufpi.edu.br

Abstract. *Based on the game Asteroids, created by Atari in 1979, this work presents a 3D version of the classic arcade title. This version maintains essentially the same mechanics as the original game, which belongs to the space shooter genre, but features ships, asteroids, and UFOs as 3D models that move through the scene in a 2D gameplay style. The ship's movement is controlled by the player, who can accelerate it and rotate it using tank-style unidirectional controls, just like in the original.*

Resumo. *Baseado no jogo Asteroids, criado pela Atari em 1979, este trabalho apresenta uma versão em 3D do clássico dos arcades. Essa versão possui basicamente o mesmo funcionamento do jogo original, que é do gênero space shooter, porém com naves, asteroides e UFOs em modelos 3D que se movimentam pela cena em jogabilidade 2D. O movimento da nave é controlado pelo jogador, que pode acelerá-la e movimentá-la para o lado em movimento unidirecional tanque, assim como no original.*

1. Introdução

Em seu contexto de criação, Asteroids foi um jogo muito popular devido a sua natureza simples e sua jogabilidade “infinita”. Além disso, o jogo inovava com a introdução da física da movimentação da nave, com a simulação da inércia do movimento no espaço sideral. Destacou-se, também, o uso de gráficos vetoriais para formação da clássica aparência do jogo, que se tornou icônica na época de seu lançamento.

Dessa forma, a popularidade e a jogabilidade divertida de Asteroids motivaram a criação deste trabalho, que é uma espécie de ressuscitação do jogo, agora com modelos e entidades tridimensionais. Para tal objetivo, o novo jogo “Asteroids 3D” aqui apresentado corresponderá as seguintes metas para conclusão:

- 1) Criação de uma nave espacial controlada pelo jogador que possa atirar, acelerar e girar o leme para a direita ou para a esquerda de modo que seja capaz de desviar dos asteroides quando fora da sua região de alcance ou que seja destruída quando colidir com os meteoros. Os tiros disparados pela nave devem ser capazes de destruir os elementos de ameaça nas cenas (UFOs e asteroides).
- 2) Criação de asteroides que, ao serem criados fora da tela, movimentem-se ao longo da cena até serem visíveis ao jogador. Esses asteroides poderão ter velocidades, tamanhos e comportamentos de rotação diferentes, surgindo em diferentes trajetórias e atravessando de uma ponta a outra da cena quando não forem destruídos. A cada certa quantidade de quadros, novos asteroides surgem e, de acordo com a pontuação do jogador, a quantidade de asteroides

aumenta e os UFOs se tornam adversários mais desafiadores.

- 3) Criação de UFOs que, periodicamente, surgem na cena e disparam tiros contra o jogador, além de serem capazes de colidir diretamente, reduzindo o número de vidas do jogador. Podem surgir em diferentes tamanhos, de acordo com a pontuação obtida pelo jogador.

2. Materiais

Para a construção das mecânicas de jogo, a linguagem C++ foi utilizada, em conjunto com as bibliotecas do Glut e do OpenGL para a geração dos moldes 3D, das funções de desenho, das funções de uso de teclado, dentre outros. Além disso, bibliotecas nativas como math, cstdlib, vector, stdio e stdlib também foram utilizadas. Todos os modelos 3D utilizados foram baixados gratuitamente em sites da internet.

Para aplicação dos modelos 3D nos moldes, a biblioteca header glm.h foi utilizada para carregar os modelos .obj baixados online. Adicionalmente, a biblioteca header stb_image foi utilizada para texturizar os objetos da cena, cujas texturas também foram baixadas gratuitamente na internet. Para áudio, a biblioteca header utilizada foi a miniaudio, que serve para reproduzir e capturar sons e funciona em múltiplas plataformas.

3. Métodos

Nesta seção, as principais funcionalidades implementadas no jogo serão descritas. São estas: Movimentação, Colisões, Inteligência Artificial, Visibilidade, Iluminação, Sombreamento, Textura, Áudio, Animação e Modelagem 3D.

Movimentação: Essa funcionalidade é exemplificada no controle do jogador, na trajetória dos asteroides e dos UFOs. Em particular, apenas a nave do jogador é controlada pelo usuário durante o jogo.

Colisões: Há colisão entre a nave do jogador e o asteroide, entre os projéteis e os inimigos (asteroide e UFO), entre os projéteis do UFO e a nave do jogador e entre a nave do jogador e os UFOs. Os demais objetos não colidem entre si.

Inteligência Artificial: O UFO foi implementado de modo que funciona como um inimigo que possui sua inteligência de rota e de tiro. Vale lembrar que ele se torna mais desafiador à medida que o jogo avança, ou seja, quando vierem UFOs grandes, estes irão atirar aleatoriamente, enquanto os menores irão dar tiros mais precisos, sempre na direção do jogador.

Visibilidade: Essa propriedade é trabalhada na definição da cena que o jogador observa, em contraste com o espaço disponível para o surgimento de asteroides e UFOs, que está fora do observável. No caso desse jogo, a cena vista pelo jogador é em modo perspectiva, com a sensação de estar observando de cima os objetos.

Iluminação: A iluminação foi adicionada ao jogo de modo que a luz vem da perspectiva do jogador, ou seja, as faces mais ocultas à visão dos jogadores aparentam ser mais sombreadas. Quando a nave faz a movimentação para o lado, percebe-se a diferença na sua iluminação, deixando de ser uniforme.

Sombreamento: Como consequência da iluminação, os objetos possuem sombra de acordo com a sua forma, sendo mais perceptível nos asteroides, pois estes fazem rotação em torno do próprio eixo. Já os UFOs, nem tanto, pois estão sempre na mesma

orientação.

Textura: Com o uso da biblioteca `stb_image`, a aplicação de texturas possibilitou uma caracterização mais distinta e de melhor visibilidade nos objetos da cena. O processo de texturização também foi utilizado para desenhar o fundo do cenário. A nave, os asteroides e o UFO são texturizados.

Áudio: Para essa funcionalidade, os sons de tiro e de explosões, somado ao som ambiente e de nitro da nave constituem o espectro de áudio do jogo, que foram importados em fontes gratuitas na internet e integrados com a biblioteca `header miniaudio`.

Animação: Para as animações, foi feita a movimentação da nave ao inclinar de um lado para o outro e o movimento giratório dos asteroides em torno de si mesmos.

Modelagem 3D: Por último, a modelagem também foi aplicada na nave, nos asteroides e nos UFOs, através de modelos com menos de 3000 polígonos, para simplificação de importação e utilização nos motores gráficos do Glut e do OpenGL.

4. Resultados e Discussão

Nesta seção, serão apresentadas as telas do jogo enquanto executando e a descrição básica do fluxo de jogabilidade, baseado em cada imagem. Na figura 1, temos o menu inicial do jogo, apenas com a mensagem de boas-vindas ditando o nome e a indicação do botão que inicia a jogabilidade e carrega os objetos na cena.

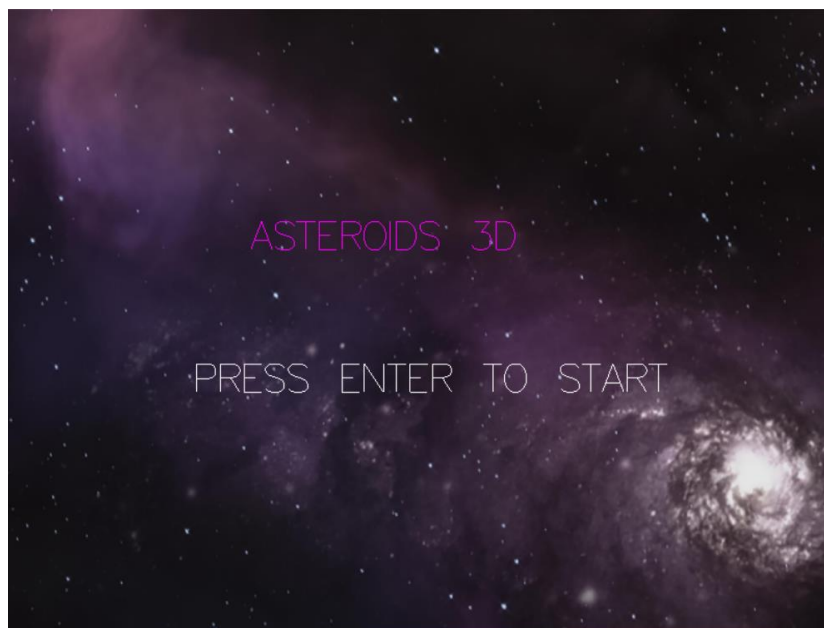


Figura 1. Tela inicial do jogo.

Na figura 2, que é a tela principal, é possível visualizar os principais elementos da interface do jogador, a qual contém seu score atual (pontuação) e seu número de vidas restantes, enquanto a cena é carregada com asteroides e UFOs que se movimentam pelo cenário e, caso acertem o jogador, reduzem em uma unidade seu número de vidas. Inclusive, é possível ver um disparo da cor vermelha, pertencente ao UFO, passando do lado esquerdo da nave do jogador.

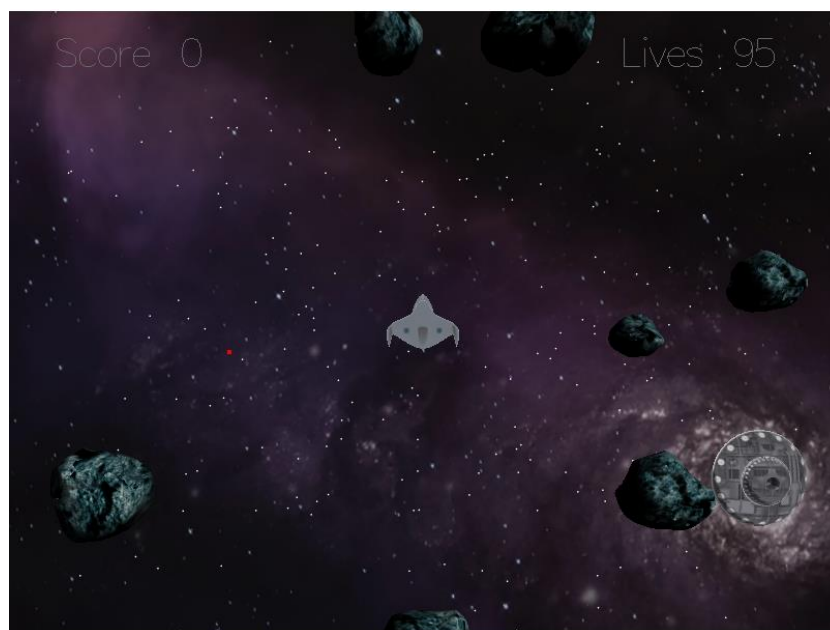


Figura 2. Visão do espaço jogável.

Em sequência, percebe-se, na figura 3, a ativação da aceleração do jogador, com o pequeno impulso azul que é projetado atrás do modelo da nave. Paralelamente, vê-se os tiros azuis provenientes do jogador e partículas de explosão geradas pelos asteroides quando destruídos, além dos fragmentos gerados pela destruição de um asteroide maior. Vale destacar que cada meteoro destruído aumenta de 100 unidades o score do jogador, que é utilizado no código para gerar mais meteoros de acordo com seu aumento e também gerar UFOs pequenos, que atiram na direção exata do jogador.



Figura 3. A comparação destes dois pontos de captura revela o aumento de 100 na pontuação (600 → 700) e a fragmentação gerada pela explosão de um meteoro maior que os demais

Já na figura 4, vemos a famosa tela de “Game Over”, atingida quando o usuário

esgota todas as suas vidas extras, perdendo o jogo e recebendo a oportunidade de recomeçar do zero.



Figura 4. Tela de fim do jogo.

5. Conclusão

Durante a criação do jogo, diversos desafios foram encontrados. Nesse sentido, vale a pena detalhar os principais empecilhos, suas causas e quais soluções foram tomadas para resolvê-los. Assim, segue os principais pontos:

- 1) Adaptação da taxa de quadros: Percebeu-se que, em máquinas diferentes, a quantidade de frames por segundo e a taxa de atualização dos objetos na cena variava bastante. Muitas vezes, algumas mecânicas de inércia e de física da nave não aparentavam funcionar em máquinas mais robustas, já que estas renderizavam as cenas com velocidade muito superior. Para resolvê-los, a taxa de atualização foi fixada e o funcionamento do jogo com uma variável de controle do tempo foi ajustado, de modo que a taxa de quadros por segundo permaneceu estável nas máquinas de teste.
- 2) Problemas de iluminação indesejada: Devido às funções de aplicação de iluminação nativas do Glut, muitas vezes a criação de textos na tela necessitava que a iluminação fosse brevemente desativada enquanto o texto fosse escrito, para que os objetos de texto não fossem influenciados pela fonte de iluminação do jogo e possuísem luz própria. Assim, alguns blocos de código tiveram de ser cercados com funções que desabilitam a luz de fontes externas, para que a cor do texto pudesse se estabelecer corretamente. A dificuldade do problema consistiu em identificar o momento certo de desabilitar ou habilitar a iluminação, sem escurecer os objetos e o fundo da cena.
- 3) Otimização de modelagem: A implementação de modelos em si não foi problema, haja vista que a biblioteca glm já possui ótimos recursos para

importação de modelos. O problema consistiu na escolha de modelos com baixa quantidade de polígonos e vértices, de modo que os objetos fossem bem aparentes e fiéis às suas representações, enquanto fossem carregados de maneira ótima pelo motor gráfico utilizado. Para resolver o problema, buscas minuciosas foram realizadas na internet em busca dos modelos gratuitos com menores quantidade de polígonos possível, para evitar travamentos e queda de performance, assim mantendo uma jogabilidade fluida.

Para trabalhos futuros, uma jogabilidade 3D poderia ser explorada, utilizando uma visão de câmera em terceira pessoa com a vista da parte traseira da nave, ou mesmo em primeira pessoa, sob a perspectiva do próprio piloto. Além disso, melhorias no algoritmo de inteligência artificial dos UFOs poderia proporcionar uma experiência mais desafiadora ao usuário. Também, um processo de estilização poderia ser incluso no processo de desenvolvimento, para trazer uma identidade mais característica ao jogo, como um menu com arte mais trabalhada, modelos próprios ou trilha sonora original.

6. Referências

Como mencionado anteriormente na primeira seção deste relatório, algumas bibliotecas foram utilizadas para a concretização do trabalho. Abaixo, segue a lista de referências utilizadas para tal fim, incluindo as documentações de cada uma e da linguagem de programação utilizada para administrá-las.

MACKRON. *miniaudio* — single-file audio playback and capture library. Repositório GitHub. Disponível em: [\[https://github.com/mackron/miniaudio\]](https://github.com/mackron/miniaudio)(<https://github.com/mackron/miniaudio>). Acesso em: 05 dez. 2025.

NOTHINGS. *stb_image.h* — single-file public-domain image loader. Repositório GitHub. Disponível em: [\[https://github.com/nothings/stb/blob/master/stb_image.h\]](https://github.com/nothings/stb/blob/master/stb_image.h)(https://github.com/nothings/stb/blob/master/stb_image.h). Acesso em: 05 dez. 2025.

G-TRUC. *GLM* — OpenGL Mathematics (C++ mathematics library for graphics software). Repositório GitHub. Disponível em: [\[https://github.com/g-truc/glm\]](https://github.com/g-truc/glm)(<https://github.com/g-truc/glm>). Acesso em: 05 dez. 2025.

CPPLANGUAGE.COM. *C++ documentation* — documentação e referência da linguagem C++. Disponível em: [\[https://cplusplus.com/doc/\]](https://cplusplus.com/doc/)(<https://cplusplus.com/doc/>). Acesso em: 05 dez. 2025.

OpenGL. *GLUT — OpenGL Utility Toolkit* — biblioteca para criação de janelas e manipulação de input/output em OpenGL. Disponível em: [\[https://www.opengl.org/resources/libraries/glut/glut_downloads.php\]](https://www.opengl.org/resources/libraries/glut/glut_downloads.php)(https://www.opengl.org/resources/libraries/glut/glut_downloads.php). Acesso em: 05 dez. 2025.